



Machine Learning Roadmap

PART 1: THE FOUNDATIONS

After mastering Python, NumPy, Pandas, and Matplotlib, you are now ready to enter one of the most exciting areas of technology: **Machine Learning (ML)**. ML allows computers to learn patterns from data and make predictions or decisions without being explicitly programmed for every scenario.



1. What is Machine Learning?



Traditional Programming

You write explicit rules that determine outcomes (e.g., "If amount > 10,000, flag as review").

Rules + Data



Output



Machine Learning

The model learns patterns from historical data to make its own predictions.

Data +
Expected
Results



Model



Note: The "Data + Expected Results → Model" formula specifically defines **Supervised Learning**. In other types of ML, the model might only receive data without expected results!



2. Main Types of Machine Learning

1

Supervised Learning

The model learns from labeled data where the desired answer (target) is already known.

Examples: Predicting house prices, classifying emails as spam, detecting fraudulent transactions.

2 Unsupervised Learning

The data does *not* contain predefined answers (unlabeled). The algorithm tries to discover hidden patterns or groups on its own.

Examples: Customer segmentation, grouping similar products for recommendations.

3 Reinforcement Learning

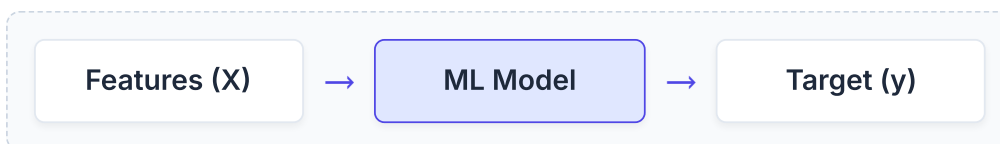
An agent learns by interacting with an environment, receiving 🏆 rewards for good actions and ❌ penalties for mistakes.

Examples: Training AI to play chess, autonomous self-driving cars, robotics.

🎯 3. Features and Target

Two essential terms you will use in every Supervised Learning project:

Term	Statistical Name	Definition	Example (Real Estate)
Features	Independent Variables (X)	The input variables used by the model to make predictions.	Square footage, Number of bedrooms, ZIP code
Target	Dependent Variable (y)	The final value we want the model to predict.	The Price of the house



🔄 4. The Complete Machine Learning Workflow

A rigorous ML project follows this step-by-step lifecycle. *Never jump straight into training a model!*

1. **Collect Data:** Gather raw historical data.
2. **Clean Data:** Handle missing values, duplicates, and outliers.
3. **Explore Data (EDA):** Visualize data to understand trends and correlations.

4. **Feature Engineering:** Select and format the best inputs (Features).
5. **Data Splitting:** Split data into *Train*, *Validation*, and *Test* sets.
6. **Train Model:** The algorithm learns from the Training set.
7. **Evaluate & Improve:** Check performance on the Validation set and tune settings.
8. **Final Test:** Do a final check on the unseen Test set to prove generalization.
9. **Predict:** Deploy the model to make predictions on entirely new data.



5. Generalization: Overfitting vs. Underfitting

The ultimate goal of ML is **Generalization**—performing well on new, unseen data.



Underfitting

Model is too simple. Fails on both training and new data.



Good Fit

Captures the underlying trend. Generalizes well to new data.



Overfitting

Memorizes training data (including noise). Fails completely on new data.



6. What is Scikit-learn?

Scikit-learn (sklearn) is Python's most popular library for *Classical* Machine Learning. It provides standardized tools for Regression, Classification, Clustering, and Data Preprocessing.

⚠ **Tech Stack Distinction:** Scikit-learn is perfect for tabular data and standard algorithms (Decision Trees, Linear Regression). However, if you want to build Deep Learning models or Neural Networks (like ChatGPT or Image Generators), you will eventually graduate to **TensorFlow** or **PyTorch**.



7. Must-Know Vocabulary Checklist

Dataset

Feature (X)

Target (y)

Training Set

Validation Set

Testing Set

Model

Prediction

Regression

Classification

Overfitting

Underfitting

Pro Tip for Beginners

Do not start Machine Learning by trying to memorize complex algorithms or complex math. First, completely understand the **Workflow** (Data → Features → Training → Evaluation). Once the architecture of how ML solves problems is clear, the individual algorithms act as simple puzzle pieces you can swap in and out.